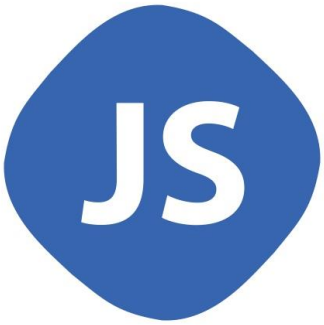


# Введение в события

- Введение
- Типы событий
- Обработчики событий
- Использование атрибута HTML
- Использование свойства DOM-объекта
- Метод addEventListener()
- События и this

## Введение

Для взаимодействия с пользователем в JavaScript определен механизм событий. Событие – это сигнал браузера о том, что что-то произошло. Все DOM-узлы умеют подавать такие сигналы (хотя события бывают и не только в DOM). Чаще всего инициатором событий является пользователь, запросивший страницу.

**События в JavaScript** — это сигналы, которые сообщают коду о различных действиях, происходящих в браузере или на веб-странице. Эти действия могут быть инициированы пользователем (например, клики мышью, нажатия клавиш) или браузером (например, загрузка страницы, изменение размера окна).

> ***Важно***. **Событие** — это не просто момент во времени, это **объект**, который **содержит информацию** о произошедшем событии.

События играют ключевую роль в интерактивности веб-страниц. Они позволяют разработчикам настраивать ответы на действия пользователя или изменения состояния браузера. Благодаря событиям, JavaScript может "слушать" эти действия и реагировать на них, выполняя определенный код.



Обработка событий является фундаментальной частью веб-разработки, так как она напрямую связана с интерактивностью и пользовательским опытом. Основные аспекты важности обработки событий включают:

- События позволяют создавать динамические, интерактивные веб-приложения. Без них веб-страницы были бы статичными и не отвечали бы на действия пользователя.
- Правильная обработка событий может значительно улучшить взаимодействие пользователя с веб-сайтом. Это включает в себя все, от базовых элементов интерфейса, таких как кнопки и формы, до более сложных взаимодействий, таких как перетаскивание элементов или жесты на сенсорных устройствах.
- События поддерживают асинхронное программирование, что особенно важно для взаимодействий, которые зависят от запросов к серверу, например, отправка формы без перезагрузки страницы.
- События позволяют управлять потоком выполнения программы, реагируя на определенные условия и действия пользователя.
- Обработка событий позволяет страницам адаптироваться к различным устройствам и размерам экрана, реагируя на события изменения размера окна или ориентации устройства.

В общем, понимание и умение эффективно работать с событиями в JavaScript критически важно для создания современных, отзывчивых и пользовательских веб-приложений.

# Типы событий

В JavaScript существует множество типов событий, каждый из которых отвечает за определенные виды взаимодействий. Ниже представлены основные категории событий и их представители.

Для ознакомления приведу список самых часто используемых DOM-событий.

### События мыши (Mouse Events)

- **click** – происходит, когда кликнули на элемент левой кнопкой мыши (на устройствах с сенсорными экранами оно происходит при касании).
- **contextmenu** – происходит, когда кликнули на элемент правой кнопкой мыши.
- **mouseover** / **mouseout** – когда мышь наводится / покидает элемент.
- **mousedown** / **mouseup** – когда нажали / отжали кнопку мыши на элементе.
- **mousemove** – движение мыши.

### События формы (Form Events)

- **submit** – пользователь отправил форму form. Обычно происходит при нажатии на кнопку отправки.
- **focus** – пользователь фокусируется на элементе, например, нажимает на input.
- **blur** – срабатывает, когда элемент теряет фокус.
- **change** – пользователь изменяет содержимое элемента, например, поля ввода input.
- **input** – событие ввода.

### Клавиатурные события (Keyboard Events)

- **keydown** – пользователь нажимает клавишу.
- **keyup** – пользователь отпускает клавишу.
- **keypress** – пользователь нажимает и отпускает клавишу. Считается устаревшим.

### События документа / окна (Document / Window Events)

- **DOMContentLoaded** – когда HTML загружен и обработан, DOM документа полностью построен и доступен.
- **load** – событие происходит, когда объект был загружен, часто используется для проверки, загрузилась ли веб-страница полностью.
- **resize** – срабатывает при изменении размеров окна браузера.



- **scroll** – зафиксируется, когда пользователь прокручивает страницу.

Каждое из этих событий имеет свои особенности и применяется в зависимости от потребностей веб-разработчика. Они позволяют создавать более интерактивные и отзывчивые веб-приложения, улучшая пользовательский опыт и обеспечивая более высокий уровень управления элементами веб-страницы.

# Обработчики событий

Событию можно назначить обработчик – некоторый **программный код** (чаще всего в виде **функции**), который сработает, как только событие **произошло**. Именно благодаря обработчикам JavaScript-код может реагировать на действия пользователя.

> ***Важно***. Есть **три способа** назначения **обработчиков событий**:
>
> - **Атрибут** HTML.
> - **DOM**-свойство.
> - Специальный метод **addEventListener**().

**HTML-атрибуты** используются редко потому, что JavaScript в HTML-теге выглядит немного странно. К тому же много кода там не напишешь.

**DOM-свойства** вполне можно использовать, но мы не можем назначить больше одного обработчика на один тип события. Во многих случаях с этим ограничением можно мириться.

**Специальный метод addEventListener** самый гибкий, однако нужно писать больше всего кода. Есть несколько типов событий, которые работают только через него, к примеру **transitionend** и **DOMContentLoaded**. Также **addEventListener** поддерживает объекты в качестве обработчиков событий. В этом случае вызывается метод объекта **handleEvent**.

Подробно метод **addEventListener** разберем отдельным уроком.



# Использование атрибута HTML

Использование обработчика в атрибуте HTML может быть выполнено в двух вариантах:

- **инлайн**-обработчик.
- **функция**-обработчик.

## Инлайн обработчики

**Инлайновые обработчики** событий назначаются непосредственно **в HTML-разметке** как атрибуты элемента. Это старый подход к обработке событий, который до сих пор используется из-за своей простоты.

> ***Важно***. Обработчик может быть назначен прямо в HTML-разметке, в **атрибуте**, который называется **on**[**событие**].

Например, чтобы назначить обработчик события **click** на элементе **input**, нужно использовать атрибут **onclick**:

```
<button onclick = "alert('Привет!')">Нажми меня</button>
```

При клике мышкой на кнопке выполнится обработчик события, код, указанный в атрибуте **onclick**.

Содержимое атрибута **onclick** должно **чередовать** двойные и одинарные кавычки. Если забыть об этом и сделать, например, так:

```
onclick = "alert("Привет!")"
```

– код работать **не будет**.

**example_1**.

```
<!-- событие документа -->
<body  onload="alert('DOM загружен!')">
```

```
<header align="center">
    <h1>События и обработка событий</h1>
</header>
<!-- ! для навешивания события используем атрибут HTML -->
<!-- событие мыши -->
<input value="Нажми меня нежно" onclick="let name='Max';
alert('Hello, ' + name + '!')" type="button"><p>
<!-- клавиатурное событие -->
<input
    id="inp"
    type="text"
    onkeyup="document.getElementById('console').innerText=doc
    ument.getElementById('inp').value"
    placeholder="Введите текст"
>
<div id="console"></div><p>
<!-- событие на элементе управления -->
<form onsubmit="alert('Сейчас будет перезагрузка
страницы!')">
    <button>Отправить форму</button>
</form>
```

Для предотвращения перезагрузки формы добавьте в обработчик строку **return false**.

```
<form onsubmit="alert('Сейчас будет перезагрузка страницы!');
return false">
    <button>Отправить форму</button>
</form>
```

Подробнее в следующих уроках.

## Функции обработчики

Атрибут HTML-тега – не самое удобное место для написания большого количества кода, поэтому гораздо удобнее создать **отдельную JavaScript-функцию**.

Следующий пример по клику запускает функцию **outMessage**().

**example_2**.

```
<!-- ! для навешивания события используем атрибут HTML -->
<button onclick="outMessage()">Нажмите, пожалуйста</button><p>
<button onclick="outMoreMessage()">Нажмите, пожалуйста</button>
<script>
    // функция-инструкция
    function outMessage() {
        alert ("Ой... А ведь здравствуйте! ;)");
    }
    // функция-стрелка
    outMoreMessage = () => {
        alert ("Виделись...");
    }
</script>
```

**example_3**.

```
<h2>Моральный кодекс строителя коммунизма:</h2>
<div id="codecs">
    <ul>
```

```
        <li>Преданность делу коммунизма, любовь к
        социалистической Родине, к странам социализма</li>
        <li>Добросовестный труд на благо общества: кто не
        работает, тот не ест</li>
        <!-- ... -->
    </ul>
</div>
<!-- ! для навешивания события используем атрибут HTML -->
<p id="but"><button onclick="hideCodecs()">Отменить
кодекс...</button></p>
<script>
    // функция управления судьбами миллионов...
    function hideCodecs() {
        let codecs = document.getElementById("codecs").innerHTML
        = "Отменено 6 декабря 1991 года, согласно декларации
        Совета Республик Верховного Совета СССР ...";
        alert ("Моральный кодекс более не актуален ...");
        but.outerHTML = "<h3>... а жаль (</h3>";
        console.log("Ничего, ничего... я тебя запомнил...");
    }
</script>
```

**example_4**.

```
    <script>
        // обработчик загрузки документа
        function fnLoadDOM() {
            // alert("Load!");
```

```
            console.log("Load!");
        }
        // обработчик изменения размеров окна
        function fnResizeWindow() {
            // alert("Resize!");
            console.log("Resize!");
        }
        // обработчик прокрутки документа
        function fnScroll() {
            alert("Scroll!");
            console.log("Scroll!");
        }
    </script>
</head>
<!-- ! для навешивания события используем атрибут HTML -->
<body
    onload="fnLoadDOM()"
    onresize="fnResizeWindow()"
    onscroll="fnScroll()"
>
    <header align="center">
        <h1>События и обработка событий</h1>
    </header>
    <ol>
        <li>Зеленая миля:
            <ul>
```

```
            <li>режиссёр: Фрэнк Дарабонт</li>
            <li>год: 1999</li>
            <li>страна: США</li>
        </ul>
    </li>
    <!-- ... -->
```

Атрибут HTML-тега **не чувствителен к регистру**, поэтому ONCLICK будет работать так же, как onClick и onCLICK… Тем не менее, **как правило**, атрибуты пишут **в нижнем регистре**: **onclick**.

## Использование свойства DOM-объекта

Можно назначать обработчик, **используя свойство DOM-элемента on**[**событие**]. Например, так, как показано в примере.

**example_5**.

```
<!-- ! для навешивания события используем свойство DOM-объекта -->
<button id="btn1">Нажмите, пожалуйста</button><p>
<button id="btn2">Нажмите, пожалуйста</button><p>
<button id="btn3">Нажмите, пожалуйста</button>
<script>
    let btn1 = document.getElementById("btn1");
    // навесим обработчики события onclick на элементы button
    btn1.onclick = function() {
        alert ("Здравствуй ТоварищЪ!");
    }
```

```
    // прямой доступ по id элемента (не рекомендуется)
    btn2.onclick = function fnHandlerClick() {
        alert ("Как поживаете??");
    }
    btn3.onclick = () => alert("Ну может хватит уже кликать без дела?")
</script>
```

Если обработчик задан через атрибут, то браузер читает HTML-разметку, создаёт новую функцию из содержимого атрибута и **записывает функцию в свойство**.

Этот способ, по сути, аналогичен предыдущему.

> ***Важно***. Обработчик всегда хранится **в свойстве DOM-объекта**, а атрибут – лишь один из способов его инициализации.

> ***Стоп***. Это важно, а вы не обратили внимание... Обработчик хранится **в свойстве DOM-объекта**. Атрибут – один из способов инициализации обработчика.

Свойству, так же, как и атрибуту, обработчиком можно назначить и уже **существующую функцию**.

Выполните, выведите в консоль и внимательно проанализируйте код следующего демонстрационного примера.

Обратите внимание, при определении обработчика мы **не используем** у функции скобки. Если поставить скобки, то произойдет **не присваивание**, а **вызов** функции при загрузке страницы. Результат вызова присвоится свойству **btn.onclick**.

Если добавить скобки, то **fnNewProperty**() – это уже **вызов функции**, результат которого (равный undefined, так как функция ничего не возвращает) будет присвоен **onclick**. Так что это не будет работать.

**example_6**.

```
<!-- ! для навешивания события используем свойство DOM-объекта -
->
<button id="btn">Нажмите, пожалуйста</button>
<script>
    // определим функцию
    function fnNewProperty() {
        alert ("Функция - это тоже свойство DOM-объекта!");
    }
    // функция по умолчанию возвращает - undefined
    // console.log(fnNewProperty()); // undefined
    /*
    такой способ неверный:
        -> btn.onclick = fnNewProperty();
        получим:
        1. fnNewProperty(); - немедленный вызов alert(...)
        2. btn.onclick = undefined;
    */
    // выводим в консоль определение функции
    console.log(fnNewProperty);
    // навесим обработчику onclick определение функции
    btn.onclick = fnNewProperty;
</script>
```



То, что я написал важно и следует красной линией через многие вопросы программирования в JavaScript. Более подробно данный материал рассматривался в уроке, посвященном замыканиям.

С точки зрения дерева DOM, следующий код выводит **атрибуты** элемента:

```
<button id="btn" onclick='alert("И снова здравствуйте!")'>Нажмите, пожалуйста</button>
<script>
    console.log(btn.getAttribute("id"));
    console.log(btn.getAttribute("onclick"));
</script>
```

Таким образом:

| Атрибут | Значение |
|---|---|
| **id** | btn |
| **onclick** | alert("И снова здравствуйте!") |

Назначить обработчик через метод **setAttribute**() **не получится**:

```
btn.setAttribute("onclick", myFun); // не сработает
```

Следующие два способа навешивания события работают одинаково.

Только **HTML**:

```
<input type="button" onclick="alert('Клик!')" value="Кнопка">
```

**HTML** + **JS**:

```
<input type="button" id="button" value="Кнопка">
<script>
    button.onclick = function() {
        alert('Клик!');
    };
```

```
</script>
```

**example_7**.

```
<!-- ! для навешивания события используем свойство DOM-объекта -
->
<button class="btn">Нажмите, пожалуйста</button><p>
<button class="btn">И здесь тоже...</button>
<script>
    // получим коллекцию элементов button
    let buttons = document.body.getElementsByClassName("btn");
    // используем анонимную и стрелочную функции
    // навесим на элементы button обработчик события onclick
    buttons[0].onclick = function() {
        alert ("И снова здравствуйте!");
    }
    buttons[1].onclick = () => {
        alert ("Вы мне надоели!");
    };
</script>
```

В следующем примере назначение обработчика **через JavaScript перезапишет** обработчик из **атрибута**.

**example_8**.

```
<button id="btn" onclick='alert("Событие - как атрибут
HTML")'>Нажмите, пожалуйста</button>
<script>
    // начальное значение атрибута onclick
    console.log("Значение атрибута onclick:");
```

```
    console.log(btn.getAttribute("onclick"));
    // начальное значение свойства onclick
    console.log("Значение свойства onclick:");
    console.log(btn.onclick);
    // перезапишем свойство onclick
    btn.onclick = () => {
        alert ("Я здесь главный!");
    }
    console.log("----------");
    // значение атрибута
    console.log("Значение атрибута onclick:");
    console.log(btn.getAttribute("onclick"));
    // значение свойства
    console.log("Значение свойства onclick:");
    console.log(btn.onclick);
</script>
```

***Важно***. Так как у элемента DOM может быть только одно свойство с именем **onclick**, то назначить **более одного обработчика таким образом нельзя** (обратите внимание на фразу - *таким образом*).

Рассмотрим еще один пример. Теперь определения функций хранятся в **массиве**. Каждый раз при перезагрузке страницы будем навешивать **случайную функцию** на обработчик события **onclick.**

**example_9**.

```
 <!-- ! для навешивания события используем свойство DOM-объекта -
->
```

```
<button id="btn">Нажмите, пожалуйста</button>
<script>
    // массив, содержащий определения функций
    let fun = [
        () => {
            alert ("Я Функция 1!")
        },
        () => {
            alert ("Я Функция 2!")
        },
        () => {
            alert ("Я Функция 3!")
        }
    ]
    // получим доступ к кнопке
    let btn = document.getElementById("btn");
    // получим случайное число
    let num = Math.floor(Math.random() * 3);
    // console.log(num);
    // каждый раз при перезагрузке страницы
    // навесим случайную функцию на обработчик события onclick
    btn.onclick = fun[num]
</script>
```

***Важно***. Убрать обработчик можно назначением **elem.onclick = null**



В демонстрационном примере **example_10** поработаем с внешними данными файла **film.js**.

**example_10. example_10.html**.

```
<!-- ! для навешивания события используем свойство DOM-объекта -
->
<!-- контейнер -->
<div id="con"></div>
<script>
    // навесим обработчик на событие onresize документа
    document.body.onload = fnOutData;
</script>
```

**example_10. script.js**.

```
// функция вывода данных массива film
function fnOutData() {
    let table = "";
    film.forEach( (obj) => {
        table += `
            <li><b>${obj.title}</b>
                <ul>
                    <li>Режиссер: ${obj.director}</li>
                    <li>Год: ${obj.year}</li>
                    <li>Страна: ${obj.country}</li>
                </ul>
            </li>
        `;
```



```
    } );
    // вставка контента в документ
    document.querySelector("#con").innerHTML =
    `<ol>${table}</ol>`;
}
```

**example_10. film.js**.

```
let film = [
    {
        title: "Зеленая миля",
        director: "Фрэнк Дарабонт",
        year: 1999,
        country: "США"
    },
    // ...
]
```

# Метод addEventListener()

Фундаментальный недостаток описанных выше способов назначения обработчика – невозможность повесить несколько обработчиков на одно событие.

Например, одна часть кода хочет при клике на кнопку делать её подсвеченной, а другая – выдавать сообщение.

Мы хотим назначить два обработчика для этого. Но новое DOM-свойство перезапишет предыдущее:

```
input.onclick = function() { alert("Привет, рад видеть!"); }
```

```
// заменим предыдущий обработчик
input.onclick = function() { alert("До свидания!"); }
```

Разработчики стандартов предложили альтернативный способ назначения обработчиков при помощи **специальных методов**:

- **addEventListener**()
- **removeEventListener**()

Эти методы свободны от указанного недостатка.

**example_11**.

```
<!-- ! для навешивания события используем метод addEventListener
-->
<button id="btn1">Умная мысль тут у меня ...</button><p>
<button id="btn2">Что что?</button><p>
<button id="btn3">Это ты мне сказал?</button>
<script>
    // для элемента btn1 навешиваем событие, назначаем обработчик
    btn1.addEventListener("click", function(){
        alert("Задача сделать человека счастливым не входила в
        план сотворения мира (З. Фрейд).");
    });
    // для элемента btn2 навешиваем событие, назначаем обработчик
    btn2.addEventListener("click", () => {
        alert("Голос интеллекта тих, но он не устает повторять —
        и слушатели находятся (З.Фрейд).");
    });
    // для элемента btn3 навешиваем событие, назначаем обработчик
    btn3.addEventListener("click", fnSayCitate);
```

```
    // функция-обработчик
    function fnSayCitate() {
        alert("Масштаб вашей личности определяется величиной
        проблемы, которая способна вас вывести из себя
        (З.Фрейд)");
    }
</script>
```

Синтаксис добавления обработчика:

```
element.addEventListener(event, handler, [options]);
```

- **event** – имя события, например "click".
- **handler** – ссылка на функцию-обработчик.
- **options** – дополнительный объект со свойствами:
- **once** – если **true**, тогда обработчик будет автоматически удалён после выполнения.
- **capture** – фаза, на которой должен сработать обработчик, фазы подробнее будут рассмотрены далее. Так исторически сложилось, что **options** может быть **false**/**true**, это то же самое, что {capture: false/true}.
- **passive** – если **true**, то указывает, что обработчик никогда не вызовет **preventDefault**(), подробнее об этом будет рассказано в главе Действия браузера по умолчанию.

**example_12**. **example_12.html**

```
<!-- ! для навешивания события используем метод addEventListener
-->
<h1>Мао Цзэдун цитаты</h1>
<div id="codecs">
    <h3>Мао́ Цзэду́н — китайский революционер, государственный,
    политический и партийный деятель XX века, главный теоретик
    маоизма, создатель современного китайского государства.</h3>
```

```
</div>
<button id="btn">Цитаты  Великого кормчего</button>
<script src="citateMao.js"></script>
<script>
    // навешиваем обработчик на событие click кнопки
    btn.addEventListener("click", () => {
        list = '';
        // перебираем элементы массив
        citate.forEach( (elem) => {
            // формируем из элементов список
            list += '<li>' + elem + '</li>';
        });
        // выводим список в документ
        btn.insertAdjacentHTML('beforebegin', '<ul>' + list +
        '</ul>');
    })
</script>
```

**example_12. citateMao.js**

```
citate = [
    'Человек, который почувствовал ветер перемен, должен строить
    не щит от ветра, а ветряную мельницу.',
    'Стремись к невозможному — достигнешь максимального.',
    // ...
]
```

***Важно***. Метод **addEventListener**() позволяет добавлять **несколько обработчиков** на одно событие элемента.

**example_13**.

```
<!-- ! для навешивания события используем метод addEventListener
-->
<input id="cmd" value="Нажми меня 3 раза, плиз..."
type="button"><p>
<script>
    // глобальная переменная
    count = 1;
    // назначаем первый обработчик событию click
    cmd.addEventListener("click", () => {
        alert('Это ' + count + ' клик');
    } )
    // назначаем второй обработчик событию click
    cmd.addEventListener("click", () => {
        if ( count++ == 3)
        cmd.outerHTML = "<h2>Все, кончилось кино!... </h2>";
    } )
</script>
```

Усложним задачу, навесим на **событие click два обработчика**, определимся с пороками и добродетелями ;)

**example_14**.

```
<!-- ! для навешивания события используем метод addEventListener
-->
```

```
<h2>Выбери жизненные принципы</h2>
<h3>Пороки</h3>
<div id="vice">
    <ul>
        <li><span>Чревоугодие</span> <input name="rb1"
        type="radio"></li>
        <li><span>Похоть</span> <input name="rb2"
        type="radio"></li>
        <!-- ... -->
    </ul>
</div>
<h3>Добродетели</h3>
<div id="good">
    <ul>
        <li><span>Умеренность</span> <input name="rb1"
        type="radio"></li>
        <li><span>Целомудрие</span> <input name="rb2"
        type="radio"></li>
        <li><span>Смирение</span> <input name="rb7"
        type="radio"></li>
        <!-- ... -->
    </ul>
</div>
<p><button id="btnControl">Сделайте свой выбор!</button>
<!-- контейнер для вывода -->
<div>
<ul id="con"></ul>
```

```
</div>
<script>
    // навешиваем обработчики на событие click элемента button
    btnControl.addEventListener("click", outList);
    btnControl.addEventListener("click", clearList);
    // обработчик Вывод выбранного
    function outList() {
        inputs = document.querySelectorAll("ul
        input[type=radio]:checked");
        if (inputs.length) {
            con.innerHTML = "";
            inputs.forEach ( (elem) => {
                con.insertAdjacentHTML("afterbegin", '<li>' +
                elem.parentNode.innerText + '</li>');
        } )
        } else
            alert("Список пуст. Определитесь!")
    }
    // обработчик Очистить список
    function clearList() {
        inputs = document.querySelectorAll("ul
        input[type=radio]");
        inputs.forEach ( (elem) => {
            elem.checked = false;
        } )
    }
</script>
```

Для удаления обработчика следует использовать **removeEventListener**:

```
element.removeEventListener(event, handler, [options]);
```

> ***Важно***. Подробнее свойство **addEventListener**() и **removeEventListener**() разберем в следующем уроке.

## События и this

Внутри обработчика события **this** ссылается на **текущий элемент**, то есть на тот, на который навешен (назначен) обработчик.

В коде ниже элемент button выводит своё содержимое, используя форму записи this.innerHTML:

```
<button onclick="alert(this.innerHTML)">Привет чудики!</button>
```

> ***Примечание***. This указывает на объект, который выполняет текущий фрагмент JavaScript-кода. This и доступ к элементу через this рассматривается в уроках:
>
> - **Область видимости и лексическое окружение**
> - **Лексическое окружение и контекст**

**example_15**.

```
<!-- будьте внимательны -->
Инлайн обработчик: <button onclick="alert(this)">Нажмите, пожалуйста</button><p>
<!-- будьте внимательны -->
Функция (инструкция) как HTML-атрибут: <button onclick="fun_1()">Нажмите, пожалуйста</button><p>
```

```
<!-- будьте внимательны -->
Функция (стрелка) как HTML-атрибут: <button
onclick="fun_2()">Нажмите, пожалуйста</button><p>
<!-- будьте внимательны -->
Функция как свойство: <button id="btn1">Нажмите,
пожалуйста</button><p>
<!-- будьте внимательны -->
Функция через addEventListener: <button id="btn2">Нажмите,
пожалуйста</button><p>
<script>
    // функция инструкция
    function fun_1 () {
        alert (this);
    }
    // функция стрелка
    fun_2 = () => {
        alert (this);
    }
    // функция-стрелка для свойства
    btn1.onclick = () => {
        alert (this);
    }
    // функция через addEventListener
    btn2.addEventListener("click", () => {
        alert (this);
    })
</script>
```

**example_16**.

```
<!-- функция с передачей this -->
Функция с передачей аргумента this: <button
onclick="fun_1(this)">Нажмите, пожалуйста</button><p>
<!-- функция без передачи this -->
Функция без передачи аргумента this: <button
onclick="fun_2()">Нажмите, пожалуйста</button><p>
<script>
    // this - ключевое слово,
    // его нельзя использовать в параметре функции
    function fun_1 (_this) {
        alert (_this);
    };
    function fun_2 () {
        alert (this);
    };
</script>
```

Не спешите запускать код демонстрационного примера **example_17**.
Проанализируйте код, попробуйте предположить вывод каждой из функций.

**example_17**.

```
<button onclick="alert(this.login)">Нажмите,
пожалуйста</button><p>
<button data-login="dataLogin"
onclick="alert(this.dataset.login)">Нажмите,
пожалуйста</button><p>
<button onclick="fun_1()">Нажмите, пожалуйста</button><p>
```

```
<button onclick="fun_2()">Нажмите, пожалуйста</button><p>
<button onclick="fun_3()">Нажмите, пожалуйста</button><p>
<script>
    // глобальная переменная login
    login = "admin";
    // функция-инструкция
    function fun_1 () {
        // изменим глобальную переменную login
        login = "globalUser";
        // обращение к глобальной переменной
        alert (this.login);
    }
    // функция-выражение
    fun_2 = function () {
        // локальная переменная login
        let login = "localUser";
        // обращение к глобальной переменной
        alert (this.login);
    }
    // функция-стрелка
    fun_3 = () => {
        // локальная переменная login
        var login = "localUser";
        // обращение к глобальной переменной
        alert (this.login);
    }
```

```
</script>
```

В коде демонстрационного примера **example_18** нет комментариев к выводу методов объекта. Разберите пример самостоятельно (свой разбор предложу ниже по тексту).

**example_18**.

```
<button onclick="alert(this.login)">Нажмите, пожалуйста</button><p>
<button onclick="obj.mtd1()">Нажмите, пожалуйста</button><p>
<button onclick="obj.mtd2()">Нажмите, пожалуйста</button><p>
<script>
    // глобальная переменная login
    login = "user";
    // объект
    let obj = {
        login: "superAdmin",
        mtd1: function () {
            // alert (this);
            alert (this.login);
        },
        mtd2: () => {
            // alert (this);
            alert (this.login);
        }
    }
</script>
```

***Примечание***. Эксперименты с кодом демонстрационного примера **example_18** продолжим в одной из **самостоятельных** работ.

**Разбор** вывода методов объекта демонстрационного примера **example_18**:

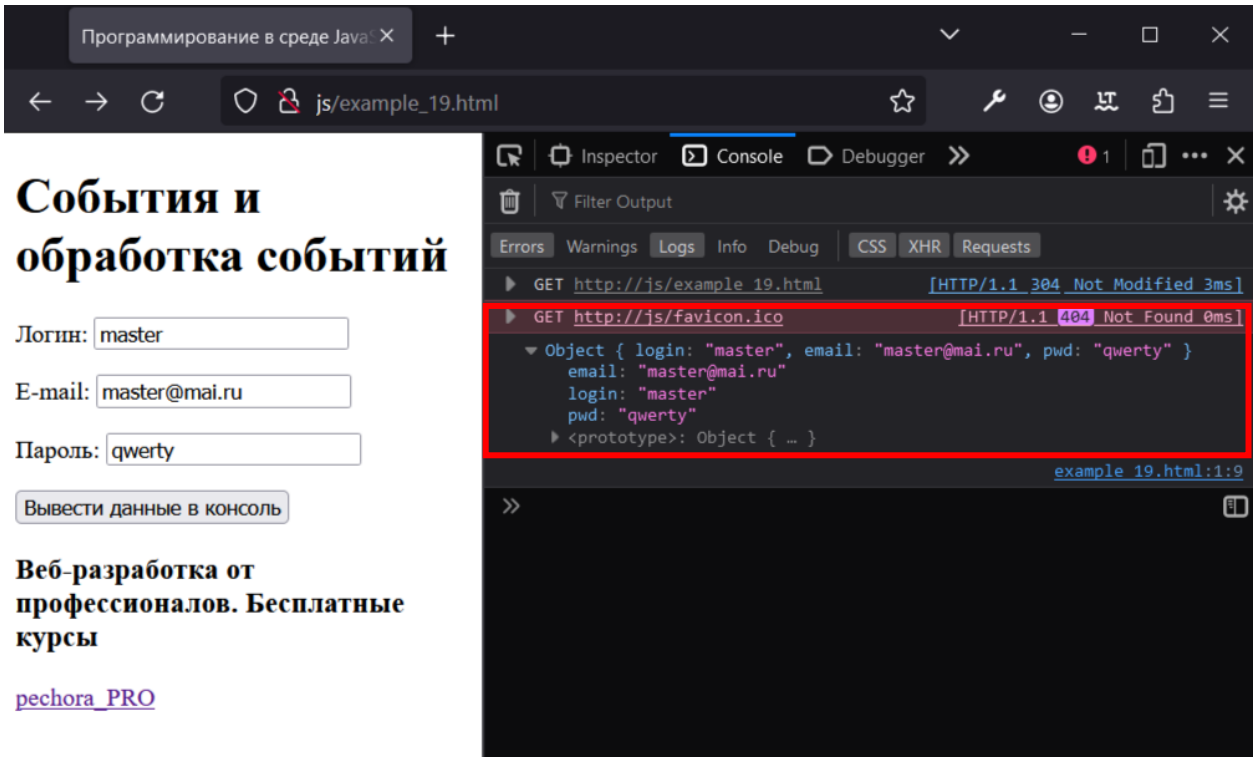
- **obj.mtd1**() – **this** в контексте объекта указывает на сам **объект**.
- **obj.mtd2**() – **this** стрелочной функции всегда **window**.

Форма – важнейший элемент разработчика приложений. Рассмотрим инлайн код создания объекта пользователя из формы.

**example_19**.

```
<script>
    // объект пользователя
    let user = {
        login: undefined,
        email: undefined,
        pwd: undefined
    };
</script>
<form onsubmit="console.log(user); return false">
    Логин:
    <input type="text" oninput="user.login=this.value"><p>
    E-mail:
    <input type="text" oninput="user.email=this.value"><p>
    Пароль:
    <input type="text" oninput="user.pwd=this.value"><p>
    <button>Вывести данные в консоль</button>
</form>
```

***Примечание***. В демонстрационном примере **example_19** создали **объект** пользователя. В одной из **самостоятельных** работ создадим **массив** объектов.



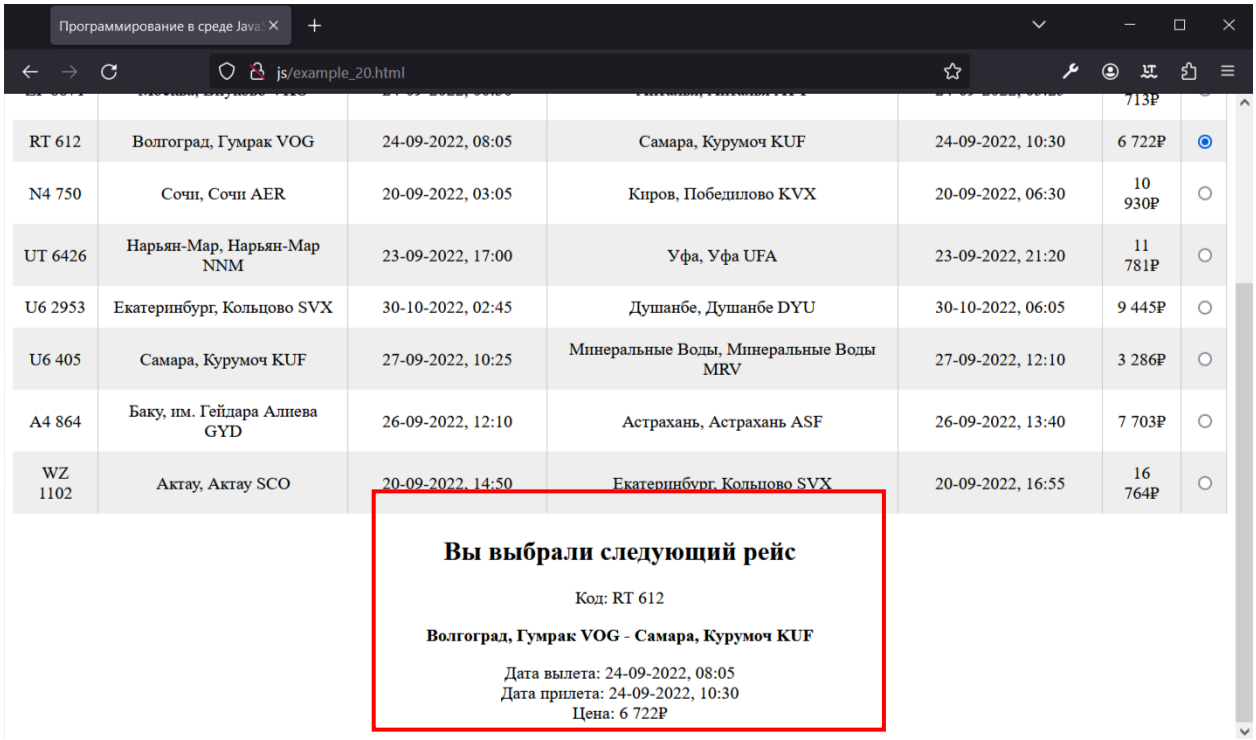

В демонстрационном примере **example_20** кликом по радиокнопке выведем информацию о выбранном рейсе.

**example_20**.

```
<div class="content">
    <table>
        <tbody>
        <tr>
            <td>Код</td>
            <td>Откуда</td>
```

```
            <td>Дата и время вылета</td>
            <td>Куда</td>
            <td>Дата и время прилета</td>
            <td>Цена</td>
            <td></td>
        </tr>
    <!-- ... -->
<script>
    // получение коллекции элементов input
    let inputs = document.querySelectorAll("table tr td input");
    // навешивание каждому элементу input обработчика события
    change
    inputs.forEach( (input) => {
        input.addEventListener("change", function() {
            // alert(this); // проверим this
            tds = this.closest("tr").children;
            out = `
                <h2>Вы выбрали следующий рейс</h3>
                Код: ${tds.item(0).innerText}  <p>
                <b> ${tds.item(1).innerText} -
                ${tds.item(3).innerText} </b><p>
                Дата вылета: ${tds.item(2).innerText} <br>
                Дата прилета: ${tds.item(4).innerText} <br>
                Цена: ${tds.item(5).innerText} <br>
            `;
            // вставка в документ сформированной строки
            document.querySelector("#console").innerHTML = out;
```

```
        })
    });
</script>
```




В одном их первых примеров мы взяли на себя смелость и отменили моральный кодекс строителя коммунизма. В последнем примере возьмем на себя наглость правки морального кодекса (ну и подведем некоторые итоги урока заодно...)

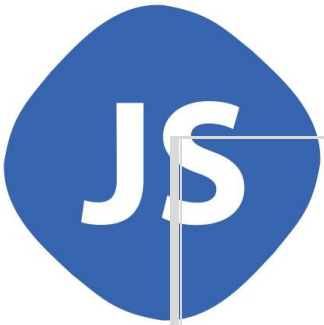
**example_21. example_21.html**

```
<!-- событие через атрибут -->
<body onload="fnLoad()">
<!-- ... -->
<h2>Правим моральный кодекс строителя коммунизма</h2>
<!-- ... -->
<div id="codecs">
    <ol>
```

```
            <li>Преданность делу коммунизма, любовь к
            социалистической Родине, к странам социализма.</li>
            <li>Добросовестный труд на благо общества: кто не
            работает, тот не ест.</li>
            <!-- ... -->
        </ol>
    </div>
    <p><button id="hide-select">Скрыть выбранное</button></p>
    <p><button id="show-hide">Простите, погорячился ...</button></p>
    <!-- подключаем функции -->
    <script src="script.js"></script>
    <script>
        // событие через свойство
        document.getElementById("hide-select").onclick =
        fnHideSelect;
        // событие через addEventListener
        document.getElementById("show-
        hide").addEventListener("click", fnShowHide);
    </script>
```

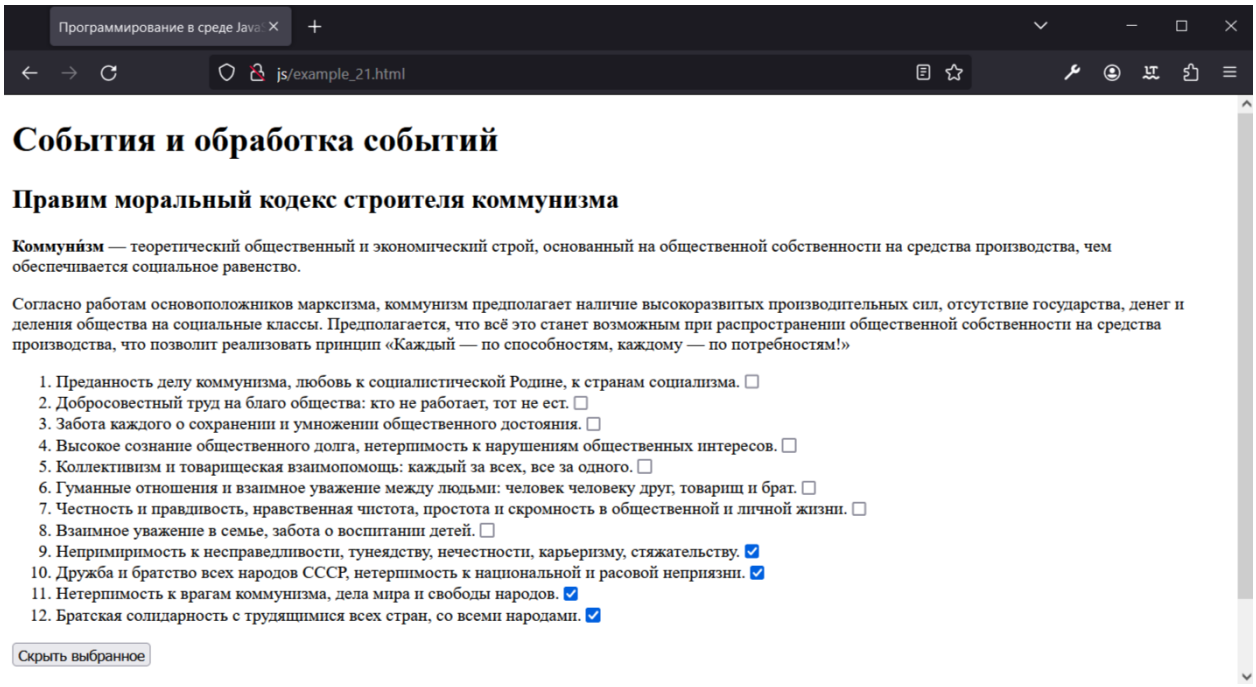
**example_21. script.js**

```
// при загрузке документа добавляем элементы checkbox к списку
function fnLoad() {
    document.getElementById("show-hide").style.display="none";
    // выбор элемментов списка
    lis = document.querySelectorAll("#codecs ol li");
    for (li of lis) {
```



```
        // добавление <input type='checkbox'> в конец каждого
        элемента списка
        li.insertAdjacentHTML("beforeend", "<input
        type='checkbox'>");
    }
}
// скрываем выбранные элементы списка
function fnHideSelect() {
    // выбор элементов списка с атрибутом checked
    inputs = document.querySelectorAll("#codecs ol li
    input:checked");
    // если есть выбранные <input type='checkbox'>
    if (inputs.length) {
        document.getElementById("show-
        hide").style.display="block";
        document.getElementById("hide-
        select").style.display="none";
        for (input of inputs) {
            // поиск и скрытие родительского элемента для
            input:checked
            input.closest("li").style.display = "none";
        }
    }
}
// отображаем исходный список
function fnShowHide() {
    document.getElementById("show-hide").style.display="none";
```

```
    document.getElementById("hide-select").style.display="block";
    // выбор элементов списка
    lis = document.querySelectorAll("#codecs ol li");
    for (li of lis) {
        // для каждого элемента списка устанавливаем display = list-item
        li.style.display = "list-item";
    }
}
```




***Примечание***. И никому не говорите, что вы правили основные положения коммунистической идеологии, не привлекайте внимание врачей, не оценят (да и небезопасно это...;)



В **самостоятельных работах** к уроку выполним следующие задания:

- Объект this в функциях и объектах.
- Поиск узла по выбранному элементу <input type="radio">. Вывод данных узла в браузер.
- Поиск коллекции узлов по выбранным элементам <input type="checkbox">. Вывод данных каждого узла в браузер.
- Создание массива объектов пользователей с использованием элемента формы (form).

---

**P.S.**

Для отработки и закрепления учебного курса **в расширенной платной версии** предоставляется следующий раздаточный материал.

К каждому уроку курса:

- Файлы **демонстрационного кода** (example).
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой теме курса:

- **Практические работы**.